Типичные ошибки реверсера и как EH/Unwind Analyzer их нивелирует

Приложение к документу «ДЕЙСТВИЯ РЕВЕРСА»: защита автоматизации ручной проверки PE32+ AMD64 EH/Unwind-метаданных

Короткий смысл документа

Этот PDF нужен для защиты проекта. Он показывает, какие типичные человеческие ошибки возникают при ручном разборе PE32+ AMD64 EH/unwind-структур и каким образом проект снижает эти ошибки через парсинг, декодирование, эвристики, динамический сборщик, отчёты и интеграцию с Ghidra.

Дата формирования: 2026-08-10 20:45

1. Главный тезис для защиты

Проект следует представлять не как обычный parser и не как malware detector. Корректная формулировка: система структурного статико-динамического анализа PE32+ AMD64 EH/Unwind-метаданных, которая автоматизирует повторяемую ручную работу реверс-инженера, снижает вероятность механических ошибок и формирует воспроизводимый evidence report.

Формула ответа комиссии

Если спросят, какие человеческие ошибки убирает проект, ответ такой: он убирает или снижает ошибки адресации, ошибок чтения PE-заголовков, ошибок mapping RVA в file offset, ошибок подсчёта RuntimeFunction entries, ошибок декодирования UNWIND_INFO, ошибок чтения handler/chain tail, ошибок интерпретации одиночных признаков, ошибок сравнения static/dynamic адресов и ошибок переноса результата в Ghidra/JSON/БД. Экспертное решение он не заменяет: он подготавливает проверяемые факты и указывает, что проверять дальше.

Именно поэтому проект корректно защищать как систему анализа: parser внутри есть, но продукт шире parser: он агрегирует факты, оценивает evidence, формирует findings, делает dynamic validation и создаёт integration artifacts.

2. Карта модулей и человеческих ошибок

Слой проекта	Какие ошибки человека снижает	Что выдаёт наружу
File/provenance	Подмена sample, потеря версии инструмента, забытые параметры запуска.	input_size, input_sha256, command_line, tool_version, schema_version.
pe.c	Ошибки RVA-to-file-offset, доверие к повреждённым headers, section OOB/overlap/misalignment.	PeImage, section list, pe_validation counters, exception directory model.
unwind.c	Неверное чтение RuntimeFunction, CountOfCodes, operands, handler tail, chain, alignment.	RuntimeFunction[], UnwindSummary[], unwind_ok/unwind_error, anomaly masks.
heuristics.c	Субъективная переоценка одиночных признаков и отсутствие единой шкалы evidence.	27 static findings, score, suspicion, confidence, weakening rules, manual review flag.
dynamic_win.c	Ошибки runtime адресации, смешение registration/hit, непонимание timeout/partial run.	DynamicReport, events, counters, trace, runtime evidence.
report.c	Потеря фактов при сериализации, invalid JSON/HTML, parsing console text.	JSON, HTML, CSV, Ghidra CSV, assessment, manual validation steps.
Ghidra scripts	Ручная разметка неверных адресов, потеря comments, слепое доверие decompiler.	Functions, comments, bookmarks, warnings по anomalies/parse failures.

3. Пять семейств evidence как защита от ручной субъективности

Семейство evidence	Какие ручные ошибки оно закрывает
table_integrity	Проверка ranges, сортировки, overlaps, duplicate begin, section structure, exception directory.
unwind_integrity	Проверка корректности UNWIND_INFO, parse failures, opcode/operand/order/alignment/chain anomalies.
metadata_placement	Проверка, где лежат unwind records и handlers: unusual, executable, writable, non-executable.
unwind_reuse	Проверка подозрительной концентрации одного UnwindInfoRVA вместо ручного подсчёта по таблице.
handler_dominance	Проверка концентрации одного handler target и контекстное подавление benign runtime-like cases.

Эти семейства важны для защиты: проект не просто перечисляет “что-то странное”, а группирует наблюдения по смысловым классам. Благодаря этому комиссия видит, что вывод строится не из одной случайной проверки, а из разных уровней структуры: таблица функций, unwind-записи, placement metadata, reuse и handler concentration.

4. Подробная карта типичных ошибок

1. Реверсер анализирует не тот файл или не фиксирует точное состояние входа

1. Идентичность файла и дисциплина воспроизводимости

Чем опасно руками. Ручной анализ легко становится невоспроизводимым: один файл открыт в Ghidra, другой запущен в отладчике, третий отправлен в отчёт. Если не зафиксировать размер, SHA-256, версию инструмента и параметры запуска, потом невозможно доказать, к каким байтам относился вывод.

Как проект нивелирует. Проект считывает файл как immutable input bytes, считает input_size и input_sha256, добавляет tool_version, report_schema_version, build_platform, build_mode и command_line в report metadata. Тем самым он связывает вывод с конкретным бинарным состоянием.

Где видно в результате. JSON: input, report_metadata. HTML: верхний блок provenance. Для backend и БД эти поля должны быть сохранены как часть analysis_job.

Что остаётся за реверсером. Это не подтверждает добросовестность источника файла. SHA-256 доказывает совпадение байтов, но не объясняет происхождение sample.

2. Путаница между байтами, числом, signed/unsigned и little-endian

2. Базовое чтение байтов и чисел

Чем опасно руками. PE поля имеют фиксированную ширину и little-endian порядок. Человек может прочитать 32-битное значение как signed, перепутать порядок байтов, принять FF FF FF FF за -1 там, где это 0xffffffff, или использовать тип C с платформозависимой шириной.

Как проект нивелирует. Внутренний слой чтения использует uint8_t, uint16_t, uint32_t, uint64_t и явные little-endian helpers. Это убирает зависимость от host-платформы и от того, как конкретный hex viewer визуально показывает байты.

Где видно в результате. Ошибки чтения не выводятся как отдельный пользовательский finding, но корректность отражается во всех RVA, размерах, section fields, RuntimeFunction entries и UNWIND_INFO summaries.

Что остаётся за реверсером. Проект не учит декодировать произвольные структуры PE вне своей области. Он аккуратно читает те поля, которые нужны для PE32+ AMD64 EH/unwind анализа.

3. Переполнение при сложении offset + size или raw_ptr + delta

3. Арифметика диапазонов

Чем опасно руками. В бинарном парсинге многие проверки выглядят просто: start + size <= file_size. Если start + size переполнилось, проверка может стать ложноположительной и привести к чтению за границей файла.

Как проект нивелирует. PE parser использует safe addition и расширение до 64-битных промежуточных значений при вычислении границ и file offsets. Подозрительные range-отношения сохраняются как structural counters и затем превращаются в findings.

Где видно в результате. JSON: pe_validation, sections, heuristic_analysis.findings с codes pe_section_raw_oob, pe_section_virtual_oob, pe_exception_directory_oob, rf_oob, unwind_rva_oob.

Что остаётся за реверсером. Безопасная арифметика защищает сам анализатор от неверного чтения. Она не доказывает, что Windows loader обязательно поведёт себя так же на каждом нестандартном файле.

4. Смешение file offset, RVA, preferred VA и runtime VA

4. Адресные системы

Чем опасно руками. Это одна из главных ошибок при ручном анализе. RVA нельзя открыть в hex viewer как file offset. Preferred VA из ImageBase нельзя автоматически сравнивать с runtime VA из отладчика, потому что ASLR может изменить actual module base.

Как проект нивелирует. Проект хранит адреса EH/unwind как RVA, переводит RVA в file offset только через section map, а в dynamic collector вычисляет runtime VA как actual loaded base + RVA. Для системных API адрес переносится через module offset, а не прямым копированием VA из процесса анализатора.

Где видно в результате. JSON: begin_rva, end_rva, unwind_rva, handler_rva; dynamic_validation.events.address/module; Ghidra CSV: begin_rva,end_rva. В отчётах адреса печатаются в hex-форме, чтобы не терять смысл битовых границ.

Что остаётся за реверсером. Пользователь всё равно должен подписывать, какой адрес он обсуждает на защите: RVA, VA или file offset. Проект снижает риск, но не отменяет терминологическую дисциплину.

5. Доверие к MZ, e_lfanew, PE signature и Optional Header без проверки границ

5. PE-заголовки

Чем опасно руками. Файл может начинаться с MZ, но не быть корректным PE32+ AMD64 image. e_lfanew может указывать за пределы файла. Optional Header может быть короче ожидаемого. Ручное чтение по фиксированным смещениям легко уводит анализ в мусор.

Как проект нивелирует. pe.c проверяет MZ, PE signature, Machine AMD64, Optional Header magic PE32+, SizeOfOptionalHeader, NumberOfRvaAndSizes и доступность нужных data directories до чтения. Неподдерживаемые форматы отклоняются как вход вне области проекта.

Где видно в результате. При ошибке CLI завершится с ошибкой чтения/парсинга. При частичных структурных проблемах они попадают в pe_validation и findings.

Что остаётся за реверсером. Проект не является универсальным PE validator для всех архитектур и не анализирует PE32, ARM64 или object files. Это осознанная граница.

6. Поиск .pdata/.xdata только по имени секции

6. Секции PE

Чем опасно руками. Имя секции в PE не является нормативной гарантией. `.pdata` и `.xdata` типичны, но toolchain может использовать другие имена, а злоумышленник может назвать секцию правдоподобно при неправильных permissions.

Как проект нивелирует. Exception Directory находится по data directory RVA, а не по имени `.pdata`. UNWIND_INFO ищется по UnwindData RVA, а не по обязательному имени `.xdata`. Имена секций используются как контекст, но permissions и диапазоны проверяются отдельно.

Где видно в результате. JSON: exception_directory, sections, function.unwind_section, findings unwind_unusual_section, unwind_in_exec_section, unwind_writable_section.

Что остаётся за реверсером. Если нестандартная секция легитимна, проект не делает автоматический вывод о вредоносности. Он помечает unusual placement и предлагает ручную проверку.

7. Игнорирование различия VirtualSize, SizeOfRawData и zero-filled области

7. Секции PE

Чем опасно руками. Часть секции может существовать в memory image, но не иметь байтов на диске. Если человек считает virtual-only область читаемой из файла, он начинает интерпретировать несуществующие bytes.

Как проект нивелирует. RVA-to-file-offset mapping разделяет принадлежность виртуальному диапазону и наличие raw-backed byte. Для чтения UNWIND_INFO требуется file-backed область, иначе декодер фиксирует parse failure или OOB-ситуацию.

Где видно в результате. JSON: pe_validation, sections.raw_size/virtual_size, unwind_error, findings rf_oob/unwind_rva_oob/unwind_parse_fail.

Что остаётся за реверсером. Static parser работает с файлом на диске. Для областей, создаваемых или меняемых при загрузке, нужен dynamic слой или ручная проверка в памяти процесса.

8. Не проверить размер exception directory и кратность записи RUNTIME_FUNCTION

8. Exception Directory

Чем опасно руками. На AMD64 каждая запись Runtime Function занимает 12 bytes. Если directory size имеет неполный хвост, ручной анализ может молча проигнорировать повреждение или сдвинуть чтение последующих записей.

Как проект нивелирует. Проект требует безопасного mapping exception directory и рассматривает неполный хвост как integrity problem, а не как нормальную ситуацию. Количество записей получается из size / 12 только после проверок.

Где видно в результате. JSON: exception_directory, runtime_function_count, pe_validation.exception_directory_oob, findings pe_exception_directory_oob.

Что остаётся за реверсером. Инструмент проверяет формат поддерживаемой модели. Он не реконструирует намерение линкера, если таблица специально повреждена.

9. Неверно понимать диапазон функции как закрытый интервал

9. RUNTIME_FUNCTION

Чем опасно руками. RUNTIME_FUNCTION задаёт полуинтервал [BeginAddress, EndAddress). Адрес EndAddress уже не принадлежит функции. Ошибка на один байт портит поиск записи для ControlPc и может создать ложный overlap.

Как проект нивелирует. Все range checks и overlap checks в проекте используют полуоткрытую модель. EndRVA должен быть больше BeginRVA, BeginRVA должен быть внутри image, EndRVA может быть равен SizeOfImage только как исключённая верхняя граница.

Где видно в результате. JSON: functions.begin_rva/end_rva/size, findings invalid_ranges, rf_oob, rf_overlaps.

Что остаётся за реверсером. Границы из `.pdata` являются metadata boundary. Соответствие реальному дизассемблированному коду всё равно нужно проверять в Ghidra при серьёзном кейсе.

10. Не заметить несортированность, duplicate begin и пересечения записей

10. RUNTIME_FUNCTION

Чем опасно руками. Таблица должна быть упорядочена по BeginAddress. Overlap или одинаковый BeginAddress нарушают однозначность: один ControlPc может попадать в несколько metadata records.

Как проект нивелирует. heuristics.c считает rf_unsorted, duplicate_begin и rf_overlaps. Overlap ищется после сортировки копии диапазонов и с учётом max_end, поэтому обнаруживаются вложенные пересечения, а не только соседние.

Где видно в результате. JSON: heuristic_analysis.findings codes rf_unsorted, duplicate_begin, rf_overlaps; details и affected_count показывают масштаб.

Что остаётся за реверсером. Проект фиксирует structural ambiguity. Почему она появилась - ошибка линкера, packer, повреждение или специальная техника - решает эксперт.

11. Считать отсутствие .pdata безусловной ошибкой

11. RUNTIME_FUNCTION

Чем опасно руками. На x64 отсутствие runtime functions может быть нормальным для leaf-only или resource-only image. Если человек автоматически объявит такой файл подозрительным, будет ложная тревога.

Как проект нивелирует. no_runtime_functions имеет medium severity и применяется с контекстом. Если executable sections отсутствуют, score не начисляется, потому что для resource-only/forwarding image отсутствие `.pdata` объяснимо.

Где видно в результате. JSON: heuristic_analysis.findings no_runtime_functions или info note в details; verdict остаётся осторожным.

Что остаётся за реверсером. Чтобы окончательно понять leaf-only случай, нужно смотреть код: есть ли calls, изменение nonvolatile state и stack frame.

12. Неверно разобрать первый байт как цельное поле вместо Version и Flags

12. UNWIND_INFO

Чем опасно руками. Первый байт UNWIND_INFO содержит bit fields: младшие 3 bits - Version, старшие 5 bits - Flags. Ручное чтение как одного числа ломает понимание EHANDLER, UHANDLER и CHAININFO.

Как проект нивелирует. unwind.c явно извлекает version и flags через masks/shifts, проверяет поддерживаемые варианты и сохраняет anomalies при неизвестной версии или противоречивой комбинации flags.

Где видно в результате. JSON: functions[].unwind.version, flags, anomalies_text; CSV: unwind_ver, unwind_flags, anomalies; Ghidra comment: UNWIND vN flags=...

Что остаётся за реверсером. Поддержка новых ABI extensions требует обновления decoder policy. Проект не обещает автоматически понимать будущие неизвестные форматы.

13. Принять CountOfCodes за число логических операций

13. UNWIND_INFO

Чем опасно руками. CountOfCodes считает двухбайтовые slots. Некоторые unwind operations занимают дополнительные slots под operands. Если считать один slot одной операцией, parser быстро съедет по структуре.

Как проект нивелирует. Decoder учитывает slot consumption для ALLOC_LARGE, SAVE_NONVOL, SAVE_XMM и других операций. Недостаток operand slots приводит к truncated/parse anomaly, а не к чтению за границей.

Где видно в результате. JSON: count_codes, fixed_stack_bytes, saved_gp, saved_xmm, anomalies_text trunc/arg/op; parse_error при невозможности безопасно продолжать.

Что остаётся за реверсером. Проект декодирует формат metadata. Он не доказывает, что каждая операция действительно соответствует машинной инструкции prolog.

14. Забыть DWORD alignment хвоста handler или chained record

14. UNWIND_INFO

Чем опасно руками. Если CountOfCodes нечётный, после массива codes есть padding slot. HandlerRVA или chained RUNTIME_FUNCTION начинается после выравнивания. Ошибка смещает чтение хвоста на 2 bytes и полностью меняет интерпретацию.

Как проект нивелирует. Decoder вычисляет tail_offset с учётом округления slots до чётного количества и помечает alignment issues. Это убирает одну из самых коварных ручных ошибок при hex-разборе UNWIND_INFO.

Где видно в результате. JSON/CSV: handler_rva, has_chain, anomalies_text alignment; Ghidra comment показывает handler или chain только если они прочитаны по правильной границе.

Что остаётся за реверсером. Language-specific data после handler RVA проект не декодирует. Он сознательно берёт общий handler pointer, а не структуры C/C++ runtime.

15. Неправильно применить масштабы operands для ALLOC/SAVE/XMM

15. UNWIND opcodes

Чем опасно руками. ALLOC_SMALL использует OpInfo * 8 + 8, ALLOC_LARGE имеет два режима, SAVE_NONVOL scale 8, SAVE_XMM scale 16. Ручная ошибка даёт неверный stack size и ложную оценку prolog.

Как проект нивелирует. unwind.c реализует правила масштабирования, собирает fixed_stack_bytes, saved_nonvol_mask и saved_xmm_mask, проверяет недопустимые OpInfo и volatile/nonvolatile sanity.

Где видно в результате. JSON: fixed_stack_bytes, saved_gp, saved_xmm, anomalies_text operand_sanity/arg; HTML functions table; Ghidra comments stack=..., saved_gp=..., saved_xmm=...

Что остаётся за реверсером. Если code намеренно расходится с metadata, структурно корректный UNWIND_INFO может пройти parser. Проверка соответствия с disassembly остаётся ручной задачей.

16. Не проверить CodeOffset относительно SizeOfProlog и порядок unwind codes

16. Prolog model

Чем опасно руками. CodeOffset должен описывать эффект инструкции prolog. Offset за пределами SizeOfProlog или нарушенный descending order делает metadata сомнительной: раскрутка может отменять то, чего не было выполнено.

Как проект нивелирует. Decoder проверяет CodeOffset <= SizeOfProlog и порядок codes. Нарушение идёт в anomalies, а не теряется как малозначимая деталь.

Где видно в результате. JSON/CSV: prolog_size, anomalies_text off/order; HTML highlights anomalous functions.

Что остаётся за реверсером. Статический проект не выполняет полный RtlVirtualUnwind для конкретного RIP/RSP. Он проверяет structural order и summary effects.

17. Автоматически считать frame pointer равным RBP или игнорировать FrameOffset

17. Frame register

Чем опасно руками. Windows x64 encoding может указать frame register и scaled FrameOffset. Если человек переносит привычку из другой модели компиляции, он неверно понимает стековый frame.

Как проект нивелирует. Decoder отдельно сохраняет frame_reg и frame_off, проверяет противоречие: FrameOffset без FrameRegister, SET_FPREG без declared register, volatile register как suspicious operand.

Где видно в результате. JSON: frame_reg, frame_off, anomalies; CSV: frame_reg, frame_off; Ghidra comment: frame=... если поле задано.

Что остаётся за реверсером. Проект не восстанавливает high-level локальные переменные. Он даёт metadata-level facts для дальнейшей проверки в disassembler/decompiler.

18. Смешать handler tail и chained RUNTIME_FUNCTION

18. Handler и ChainInfo

Чем опасно руками. EHANDLER/UHANDLER означает HandlerRVA. CHAININFO означает тройку chained RUNTIME_FUNCTION. Комбинация CHAININFO с handler flags противоречит формату. Ручной разбор может выбрать неверный смысл хвоста.

Как проект нивелирует. Decoder различает handler tail и chain tail, проверяет invalid chain flag combinations, сохраняет has_handler/handler_rva или has_chain/chain fields отдельно.

Где видно в результате. JSON: has_handler, handler_rva, has_chain, chain; anomalies_text chain_invalid; Ghidra comment: handler_rva=... или chain_to=...

Что остаётся за реверсером. Проект не декодирует C scope tables, C++ EH records и compiler-specific language data после handler RVA.

19. Следовать chain бесконечно или без проверки циклов

19. Chained unwind

Чем опасно руками. Специально созданный файл может образовать self-loop или длинную цепочку. Ручной анализ может зависнуть концептуально, а наивный parser - по ресурсам.

Как проект нивелирует. Проект поддерживает follow_chain с max_chain_depth, обнаруживает self-loop/visited RVA loop и помечает chain_depth/chain_loop anomalies. CLI ограничивает max-chain-depth допустимым диапазоном.

Где видно в результате. JSON: chain, anomalies_text chain_loop/chain_depth; report_metadata: follow_chain, max_chain_depth.

Что остаётся за реверсером. Ограничение глубины - защитная policy анализатора, а не нормативное свойство Windows. При редких legitimate длинных chain нужна ручная проверка.

20. Не проверить, куда указывает HandlerRVA

20. Handler placement

Чем опасно руками. Handler должен указывать на исполнимый код. RVA в non-executable, unknown или writable section требует внимания. Человек может увидеть сам факт handler и не проверить section permissions.

Как проект нивелирует. heuristics.c проверяет handler section: non-executable/unknown, writable placement и dominance одного handler среди множества records. Комбинации с writable metadata усиливают сигнал.

Где видно в результате. Findings: handler_nonexec_section, handler_writable_section, handler_dominance, writable_metadata_handler_combo, placement_exec_nonexec_handler_combo.

Что остаётся за реверсером. Section characteristics из файла не всегда равны runtime page protections. Для сильного вывода нужно сопоставить с динамикой или memory map.

21. Либо игнорировать массовый reuse UnwindInfoRVA, либо считать любой reuse вредным

21. Повторное использование metadata

Чем опасно руками. Некоторое переиспользование unwind metadata бывает нормальным. Но слишком высокая концентрация одного UnwindInfoRVA или одного handler может указывать на packer/runtime/обфускацию.

Как проект нивелирует. Проект считает unwind_reuse_ratio, unwind_unique_ratio и handler_reuse_ratio. Strong reuse требует n > 200 и ratio > 0.30. Moderate reuse требует corroboration другим static signal. Dominant handler подавляется в benign runtime-like context.

Где видно в результате. JSON: reuse ratios, findings unwind_reuse_moderate, unwind_reuse_strong, handler_dominance, reuse_dominance_combo; weakening rules показывают осторожность.

Что остаётся за реверсером. Reuse - эвристика, а не доказательство. На защите правильно говорить: проект выявляет концентрацию evidence, а не автоматически классифицирует происхождение.

22. Приравнять score/confidence к вероятности вредоносности

22. Эвристики

Чем опасно руками. Комиссия может специально спросить: точность модели? Если назвать score вероятностью, это будет методологическая ошибка. Вес 25 не является стандартом Microsoft и не получен статистической калибровкой.

Как проект нивелирует. В проекте score - bounded engineering score, suspicion - score/100, confidence - rule-based уверенность в полноте и взаимном подтверждении evidence. Verdict формулируется осторожно: no_strong_static_anomalies, low/moderate/high suspicion.

Где видно в результате. JSON: heuristic_analysis.score, suspicion, confidence, verdict, evidence_level, weakening_rules_applied; docs/evidence_matrix.md объясняет веса и границы.

Что остаётся за реверсером. Без размеченного реального корпуса нельзя честно заявлять статистическую accuracy. Можно заявлять воспроизводимость, покрытие synthetic cases и инженерную объяснимость правил.

23. Сделать вывод по одному признаку без контекста

23. Эвристики

Чем опасно руками. Один unusual section name, один маленький sample или один weak signal может иметь benign объяснение. Реверсер может переоценить единичное наблюдение.

Как проект нивелирует. Heuristic engine применяет weakening rules: single_signal_only, small_sample_size, placement_only_signal, single_family_only. При этом факт не удаляется, а confidence снижается. Это сохраняет evidence и показывает осторожность.

Где видно в результате. JSON: weakening_rules_applied, confidence, requires_manual_review; HTML: блок weakening rules.

Что остаётся за реверсером. Mitigation не отменяет finding. Оно говорит: автоматический вывод менее уверенный, нужна ручная проверка.

24. Потерять parse failure или заменить неизвестное значение красивым числом

24. Отчётность

Чем опасно руками. Если парсер не смог безопасно прочитать UNWIND_INFO, нельзя подставлять нули как будто это реальные поля. Иначе downstream analyst примет недостоверные значения за факты.

Как проект нивелирует. При parse failure CSV пишет NA для ненадёжных полей и сохраняет parse_error. JSON держит unwind_ok=false и unwind_error, но сохраняет форму массива functions. HTML показывает failure отдельно.

Где видно в результате. CSV: NA/parse_error; JSON: functions[].unwind_ok, unwind_error; findings unwind_parse_fail; Ghidra comment: UNWIND parse failed: ...

Что остаётся за реверсером. Пользователь должен читать parse_error как ограничение данных. Не все строки отчёта имеют одинаковую доказательную силу.

25. Парсить console output вместо JSON-контракта

25. Отчётность

Чем опасно руками. Консольный summary удобен человеку, но не является стабильным API. Если backend будет выдирать строки из terminal output, малое изменение текста сломает интеграцию.

Как проект нивелирует. Проект выдаёт machine-readable JSON со schema version и docs/report.schema.json. Report writer проверяет metadata, escaping, model bounds и пишет файлы атомарно.

Где видно в результате. JSON report, report.schema.json, report_metadata.report_schema_version; HTML/CSV являются дополнительными outputs.

Что остаётся за реверсером. Для production backend нужно валидировать JSON schema и хранить raw_json. Не следует строить бизнес-логику на английских display messages.

26. Слепо довериться decompiler output или автоматическим boundaries

26. Ghidra

Чем опасно руками. Ghidra создаёт реконструкцию: function boundaries, types, decompiler C-like text. Если EH/unwind metadata повреждена, imported function map тоже может быть неверной. Автоматическая разметка - не доказательство.

Как проект нивелирует. Ghidra CSV передаёт begin_rva, end_rva, synthetic name и compact unwind comment. Import script старается не перетирать пользовательские имена, добавляет comments/bookmarks и помечает parse failures/anomalies.

Где видно в результате. Ghidra CSV columns begin_rva,end_rva,name,comment; import_unwind_csv.py; bookmarks warning при anomalies/parse failed; comments с prefix [EH/Unwind Analyzer].

Что остаётся за реверсером. После импорта нужно сверять findings table, prolog instructions и handler disassembly. Ghidra-слой ускоряет навигацию, но не заменяет экспертную проверку.

27. Считать динамику доступной на любой ОС

27. Dynamic layer

Чем опасно руками. Статический анализ PE-файла возможен на macOS/Linux/Windows. Но dynamic collector использует Windows Debug API и должен запускать Windows x64 target. Попытка ожидать полноценную динамику на macOS или Linux без отдельной Windows-среды неверна.

Как проект нивелирует. Код имеет Windows branch и non-Windows stub. На неподдерживаемой платформе dynamic report явно сообщает ошибку, а не имитирует исполнение.

Где видно в результате. dynamic_validation.enabled/ran/ok/timed_out/details; на non-Windows ran=false/ok=false с понятным сообщением.

Что остаётся за реверсером. Для защиты: статическое ядро кроссплатформенно как анализатор PE bytes, динамическое подтверждение требует Windows x64 host или отдельный Windows worker.

28. Запускать неизвестный binary без изоляции

28. Dynamic safety

Чем опасно руками. Dynamic mode реально исполняет target. Target может обращаться к файлам, сети, создавать процессы, зависать или вести себя иначе под debugger. Timeout и DEBUG_ONLY_THIS_PROCESS не являются sandbox.

Как проект нивелирует. Collector ограничивает timeout, event arrays, trace steps, фиксирует partial session status и не заявляет обход защит. Документация прямо требует disposable Windows VM/hardened sandbox для production deployment.

Где видно в результате. dynamic_validation.timeout_ms, timed_out, target_exited, events, dropped_events; manual validation steps могут рекомендовать dynamic run.

Что остаётся за реверсером. Без внешней sandbox-инфраструктуры проект не делает запуск неизвестных файлов безопасным. Это должен обеспечивать backend/операционная среда.

29. Ставить breakpoint по адресу из процесса анализатора или по preferred ImageBase

29. Dynamic addresses

Чем опасно руками. ASLR делает bases разными. Адрес RtlAddFunctionTable в analyzer process не равен адресу в target process. HandlerVA нельзя считать как preferred ImageBase + RVA, если target загружен по другому base.

Как проект нивелирует. Collector получает actual base из CREATE_PROCESS_DEBUG_EVENT. Static handler breakpoint ставится как base_main + HandlerRVA. System API breakpoint переносится через local symbol offset относительно actual module base target.

Где видно в результате. dynamic details self-check, module formatting, events.address/module; counters API registrations and handler hits.

Что остаётся за реверсером. Если target использует другую версию DLL, module substitution или нестандартный loader scenario, remote export parsing был бы точнее. Текущий подход рассчитан на обычную Windows-среду.

30. Неверно трактовать first-chance, second-chance, UEF и timeout

30. Dynamic events

Чем опасно руками. First-chance exception ещё не означает сбой программы. UEF под активным debugger может вести себя не так, как без debugger. Timeout не равен clean run. Partial dynamic events нельзя выбрасывать, но нельзя показывать как complete success.

Как проект нивелирует. DynamicReport разделяет ran, ok, timed_out, target_exited, target_exit_code, first_chance_seen, second_chance_seen. Report layer сохраняет partial events даже при ok=false и ограничивает runtime evidence incomplete session.

Где видно в результате. JSON dynamic_validation: ok/timed_out/target_exited/counters/events/details; assessment.case_strength учитывает incomplete status осторожно.

Что остаётся за реверсером. Динамика наблюдает только конкретный execution path с конкретными аргументами и окружением. Отсутствие event в одном запуске не доказывает, что механизма нет вообще.

31. Не различать регистрацию handler и фактическое попадание в handler

31. VEH/VCH/UEF

Чем опасно руками. Вызов AddVectoredExceptionHandler показывает, что программа передала pointer в API. Это ещё не доказывает, что handler был успешно зарегистрирован и реально сработал.

Как проект нивелирует. Collector сохраняет separate counters: registrations и hits для VEH, VCH, UEF, а также static unwind handler hits. Это не смешивается в одну строку.

Где видно в результате. dynamic_validation.counters: veh_registrations/veh_hits, vch_registrations/vch_hits, uef_set_calls/uef_hits, unwind_handler_hits; events veh_registration/veh_handler_hit и т.д.

Что остаётся за реверсером. Текущий collector не ставит return breakpoint на registration API, поэтому не знает return value. Hit handler сильнее, чем сам факт вызова регистрации.

32. Думать, что проект полностью разбирает runtime-generated tables

32. Dynamic function tables и JIT

Чем опасно руками. JIT и runtime могут создавать function tables при исполнении через RtlAddFunctionTable или RtlInstallFunctionTableCallback. Простая статическая проверка файла их не увидит, а динамический hit API ещё не раскрывает содержимое таблицы.

Как проект нивелирует. Collector фиксирует вызовы RtlAddFunctionTable и RtlInstallFunctionTableCallback, аргументы table/count/base/callback, повышает runtime evidence и может коррелировать это с placement anomalies.

Где видно в результате. events rtl_add_function_table, rtl_install_function_table_callback; dynamic_signal_count; final assessment correlation для placement anomaly + runtime function-table activity.

Что остаётся за реверсером. Текущая реализация не читает remote RUNTIME_FUNCTION array и не декодирует callback result. Поэтому на защите честно: проект подтверждает runtime registration activity, но не выполняет полный JIT table decoder.

33. Считать динамику автоматическим оправданием или автоматическим обвинением статики

33. Static/dynamic correlation

Чем опасно руками. Если статически metadata writable, а динамика показала RtlAddFunctionTable, это может объяснять runtime-сценарий, но не доказывает benign. Если динамика ничего не показала, это может быть просто недостигнутый path.

Как проект нивелирует. Assessment использует conservative correlation: suspicious handler context + handler/VEH/VCH/UEF hits или placement anomalies + dynamic function-table activity. Dynamic events не меняют static score, а добавляют evidence level, case_strength и confidence context.

Где видно в результате. assessment.has_static_dynamic_correlation, static_evidence_level, runtime_evidence_level, case_strength, final_verdict/final_confidence.

Что остаётся за реверсером. Correlation сейчас агрегированная, не per-function proof graph. Для строгого вывода нужно address-level join и ручная проверка конкретных RVA.

34. Сравнивать absolute runtime VA между запусками или потерять raw JSON

34. Backend, БД и сравнение анализов

Чем опасно руками. Из-за ASLR абсолютные адреса меняются между runs. Если БД хранит только normalized verdict и выбрасывает raw_json, потом нельзя переинтерпретировать старый отчёт при смене policy.

Как проект нивелирует. Report design отдаёт stable keys: input_sha256, tool_version, schema_version, begin_rva, finding code, representative_rva, event type + module + offset/RVA. В документации рекомендовано хранить raw_json как source of truth.

Где видно в результате. JSON report, report_metadata, functions, findings, dynamic events. Backend должен использовать subprocess boundary и schema validation.

Что остаётся за реверсером. Сама БД и API - внешний слой. Ядро даёт корректный машинный контракт, но storage policy должен реализовать backend-разработчик.

5. Как это презентовать на защите

Ниже - готовая логика ответа, если комиссия спросит, зачем проект нужен, если реверсер и так может открыть PE в Ghidra, hex viewer и debugger.

• Проект автоматизирует не профессию реверсера целиком, а повторяемую и ошибкоопасную проверку PE32+ AMD64 EH/unwind metadata.

• Главная ценность - не только скорость, а воспроизводимость: одинаковые bytes и одинаковые параметры дают один и тот же структурный отчёт.

• Парсерный слой убирает механические ошибки чтения: endian, width, bounds, RVA mapping, section ranges, table size.

• Decoder убирает ошибки формата UNWIND_INFO: slot counting, tail alignment, operands, flags, chain.

• Heuristic layer снижает субъективность: findings имеют code, family, severity, affected_count, representative RVA, score и weakening rules.

• Dynamic layer не заменяет статику. Он добавляет runtime evidence: регистрация VEH/VCH/UEF, hits handlers, dynamic function table activity, exception flow.

• Ghidra layer не делает магическую деобфускацию. Он переносит факты в рабочую среду реверсера: function boundaries, comments, bookmarks and warnings.

• Ограничения названы явно: нет полной декомпиляции языка, нет malware verdict, нет гарантии покрытия всех runtime paths, нет статистической probability без размеченного corpus.

6. Что проект принципиально не обещает

• Он не доказывает вредоносность файла. Он выявляет структурные anomalies и runtime evidence вокруг EH/unwind metadata.

• Он не заменяет disassembly и decompiler review. Особенно важно вручную сопоставлять UNWIND_INFO с фактическим prolog code.

• Он не декодирует полностью C/C++ language-specific exception data после handler RVA.

• Он не является полноценным JIT table decoder: dynamic layer фиксирует runtime function table registration, но не читает и не валидирует весь remote table content.

• Он не sandbox. Dynamic mode требует изолированной Windows-среды.

• Он не выдаёт статистическую probability. Score и confidence - engineering assessment, а не математически откалиброванная модель.

• Он не гарантирует покрытие всех execution paths. Dynamic events относятся к конкретному запуску с конкретными аргументами и окружением.

7. Сильная защитная формулировка

Сильный ответ звучит так: ручной анализ EH/unwind metadata требует большого числа однотипных, но точных действий: различать RVA/VA/file offset, проверять PE ranges, правильно считать RuntimeFunction entries, декодировать variable-length UNWIND_CODE array, учитывать DWORD alignment, не путать handler и chain tail, проверять section permissions, затем переносить результат в Ghidra и сопоставлять со статикой/динамикой. Вручную на каждом из этих шагов возможны ошибки. Проект превращает эту цепочку в воспроизводимый pipeline, сохраняет raw facts, формирует explainable findings, указывает confidence limitations и оставляет эксперту именно экспертную часть: интерпретацию назначения кода и проверку спорных случаев.

8. Опора на материалы проекта

Документ является обобщением теории и реализации проекта. Он не добавляет новые правила анализа поверх ядра, а объясняет, какие человеческие ошибки закрываются уже существующими слоями.

• docs/ru/итогоая теория.pdf: теория PE, RVA/VA/file offset, Windows x64 EH/unwind, Ghidra, static/dynamic analysis.

• book_source/teacher_book/02_data_memory_addresses.md: адресные системы, ASLR, endian, overflow.

• book_source/teacher_book/05_pe_format_as_loader_contract.md: PE/COFF, section mapping, exception directory.

• book_source/teacher_book/06_exception_dispatch_unwind_theory.md: RUNTIME_FUNCTION, UNWIND_INFO, handler tail, chaining.

• book_source/teacher_book/07_reverse_engineering_method_ghidra_debugger.md: метод реверса, Ghidra, дисциплина вывода.

• book_source/teacher_book/15_heuristics_evidence_engine.md: findings, score, confidence, weakening rules.

• book_source/teacher_book/16_dynamic_windows_collector.md: Windows Debug API, VEH/VCH/UEF, dynamic function tables.

• book_source/teacher_book/17_reporting_assessment_integration.md: JSON/HTML/CSV/Ghidra CSV, schema, backend contract.

• docs/evidence_matrix.md: пять семейств evidence и 27 статических finding-кодов.